

CS6004 PA4

Report

Jigyasa Chouhan, Yash Sabale
23B1083, 23B1043

1 Optimization Description

1.1 What was implemented

We have implemented a static-analysis based optimization using the Soot framework. The core idea was to analyze the perform a points-to-analysis to de-virtualize method calls at appropriate call sites. The characteristics of the optimization are as follows:

- At a call site, if we can infer that the object on which the method is dispatched can be of only one type/class, we can use that information to optimize the virtual table look up occurring at this call site, since we now know the target.
- If the target method is small in size (customizable threshold for size), we are inlining the method at the call site.
- If the target method is too big to be inlined (inlining big methods causes the code to get bloated), we generate a static version of the target method called `methodNameStatic` and replace the original virtual invoke at the call site with a static invoke to the newly made static target.
- After each iteration of inlining/staticizing, we run a dead method elimination process that deletes all methods that are now unreachable from the main function.
- The iterative optimizations are terminated when there is an iteration with no inlining or staticizing possible ie fixed point is found.

The implementation consists of:

- `Analysis.java`: Performs the core analysis over methods.
- `AnalysisTransformer.java`: Integrates the analysis into Soot's transformation pipeline : inlining/staticizing and dead method elimination iteratively.

- `Pgraph.java`: Represents the points to graph abstraction used for analysis
- `PA4.java`: Driver code to run the analysis and transformation.

1.2 Why it is useful

This optimization is useful because:

- Method inlining is efficient because it removes the two level indirection that occurs in case of a virtual call to a function - vTable look up and jump to the target method.
- Inlining large methods at every call site can blow up the size of the code, thus the customizable threshold can be used to limit the size of the methods being inlined.
- Static invokes are faster than virtual invokes since there is no virtual table lookup involved in the former. It reduces one level of redirection.
- Deleting unreachable methods reduces the size of the code significantly.

Overall, the optimization improves both analysis efficiency, memory consumption and runtime performance of transformed programs.

2 Analyses Used

2.1 Dimensions of Precision

The analysis incorporates several precision dimensions:

- **Flow-sensitive**: We are using the Forward Flow Analysis framework to generate Points-to graphs for each method, which internally performs a flow sensitive analysis to obtain the possible objects a variable or field could point to at each statement in the method.
- **Context-insensitive**: Method calls are analyzed without distinguishing between different call sites.
- **Intraprocedural**: The points-to analysis does not use the analysis results of other methods which are called inside the method being considered. We perform a conservative update at method invokes, assuming no information about them.
- **Termination**: Since the transformation is iterative and is stopped only when a fixed point is reached, it is important to argue that a fixed point is always attained and the algorithm terminates.
We can argue this because inlining will stop once all methods grow beyond the threshold size, which is finite. Once nothing can be inlined, all methods are eligible only for static call replacement, since there are only finite call sites that are never increased due to the staticization, this process will definitely terminate.

2.1.1 Assumptions

- Static calls are always faster than virtual calls
- Static calls are fast enough and are not being optimized at all
- All variables are initialised from every path of the call graph
- Test case does not contain any interface

3 Transformation Details

3.1 Before vs After Transformation

3.1.1 Inlining

Originally, the methods were being called from `Test::run()`, but since we know the exact target of the call sites, they have been inlined to perform the method body in the caller method itself

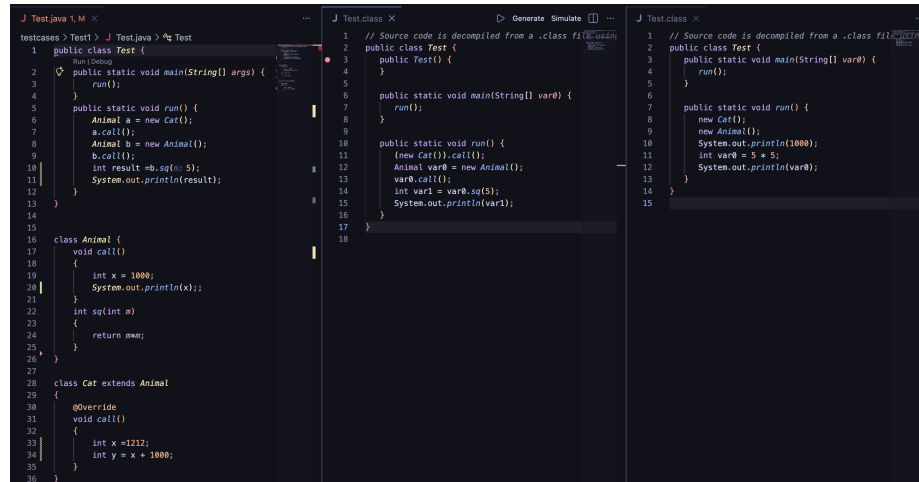


Figure 1: Inlining

3.1.2 Static Call Replacement

`Animal::call()` is a big method so it is not eligible for being inlined, hence we have made a static version called `call_static`, that is being called in the caller instead of the virtual call.

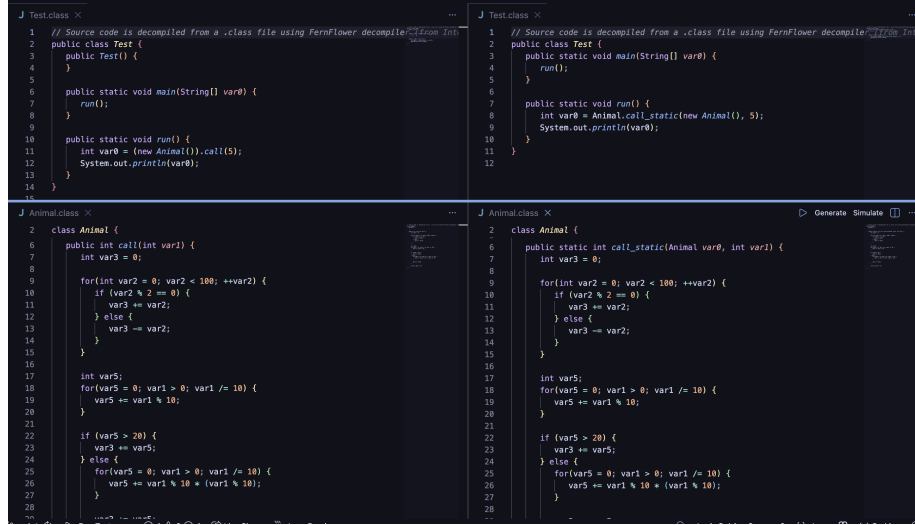


Figure 2: Static Call Replacement

3.1.3 Dead method elimination

Although this is not really an object oriented optimization, we have performed this since it saves significant code memory, also makes the transformations/analyses faster reducing the load for each iteration. As visible, since `Animal::call()` was replaced by `Animal::call_static()`, and the original method was not being called anywhere else, we can remove the method.



Figure 3: Dead code elimination

3.2 Key Challenges

Several challenges were encountered:

- **Handling recursive components:** Ensuring correctness when methods are mutually recursive required us to limit the size of methods for inlining to a finite number.
- **Interprocedural Points-to Information loss:** Since our analyses is not interprocedural, there are cases wherein monomorphization opportunities are lost due to imprecision of the analyses.

- **Duplicate functions:** Duplicate functions with different names may or may not be detected since the analysis depends on the order in which the methods are processed. Thus, some duplicates might still be present at the end.

4 Result Tables

Test ID	Original Time (nanosec)	Optimized Time (nanosec)	% Improvement
1	1999398	1944172	2.76
2	2681687	2148144	19.89
3	42874	24654	42.49
4	70490	44268	37.19
5	177191	115900	34.59
6	37530	27215	27.48
7	43777	38873	11.20
8	1967785	1883044	4.3
9	2159097	2087821	3.3
10	28925	26708	7.6

Table 1: Performance comparison between original and optimized code